

CTF 7 — Stop Putting Everything on One Screen

Goal: Introduce Angular routing and separate the Student Portal into dedicated pages.

School Request

"Each student deserves a dedicated page."

Today's mission introduces **Angular Router**. Instead of keeping the entire Student Portal on one screen, you'll create separate pages for the student list, student details, and adding a new student.

You'll also practice navigating between pages using Angular routes.

Flag 1 — Prepare Your Mission

Objective: Create a new branch for today's challenge.

Create and switch to a new Git branch.

Branch name:

feature/routing



Screenshot:

- The command used to create and switch to the new branch.

```
Student@COLLEGEPC-9 MINGW64 ~/my-first-angular-app (feature/conditional-ui)
● $ git checkout -b feature/routing
Switched to a new branch 'feature/routing'

Student@COLLEGEPC-9 MINGW64 ~/my-first-angular-app (feature/routing)
○ $
```

Flag 2 — Build the Student List Page

Objective: Create a dedicated page for the student list.

Move your existing student list UI into a separate component/page.

Your page should display:

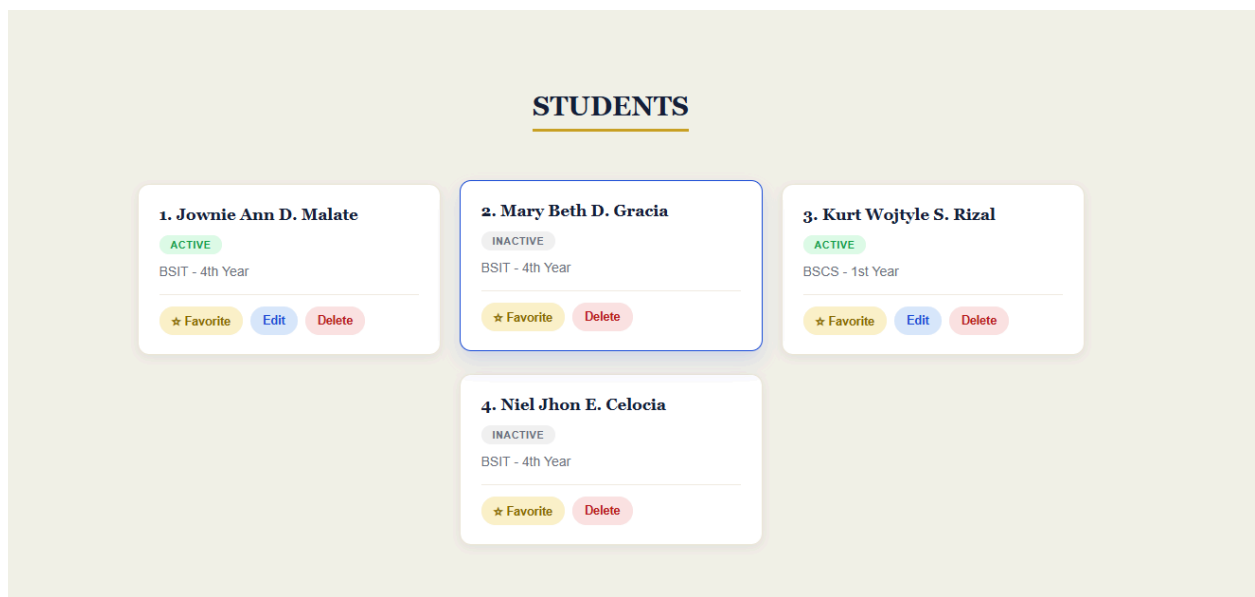
- Student cards
- Student information
- Active/Inactive status
- Existing actions

The Student List page will become the **main page** of the Student Portal.



Screenshot:

- Your Student List page displayed in the browser.



Flag 3 — Create the Student Detail Page

Objective: Create a separate page for viewing a student.

Create a new component for the Student Detail page.

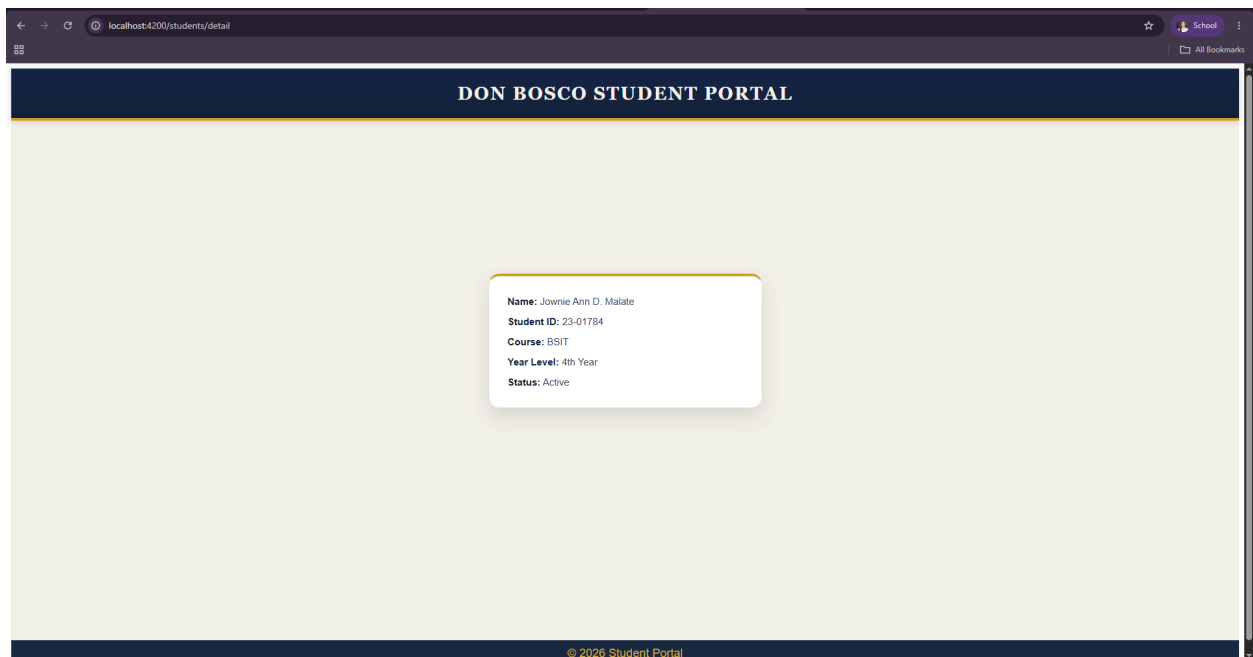
For now, the page can display information such as:

- Student name
- Student ID
- Course
- Year level
- Status

You will later connect this page to a specific student.

 **Screenshot:**

- Your Student Detail page displayed separately from the Student List.



Flag 4 — Create the Add Student Page

Objective: Create a dedicated page for adding students.

Create another component for:

Add Student

For this challenge, **do not create any form fields yet**.

The page can simply display:

Add Student

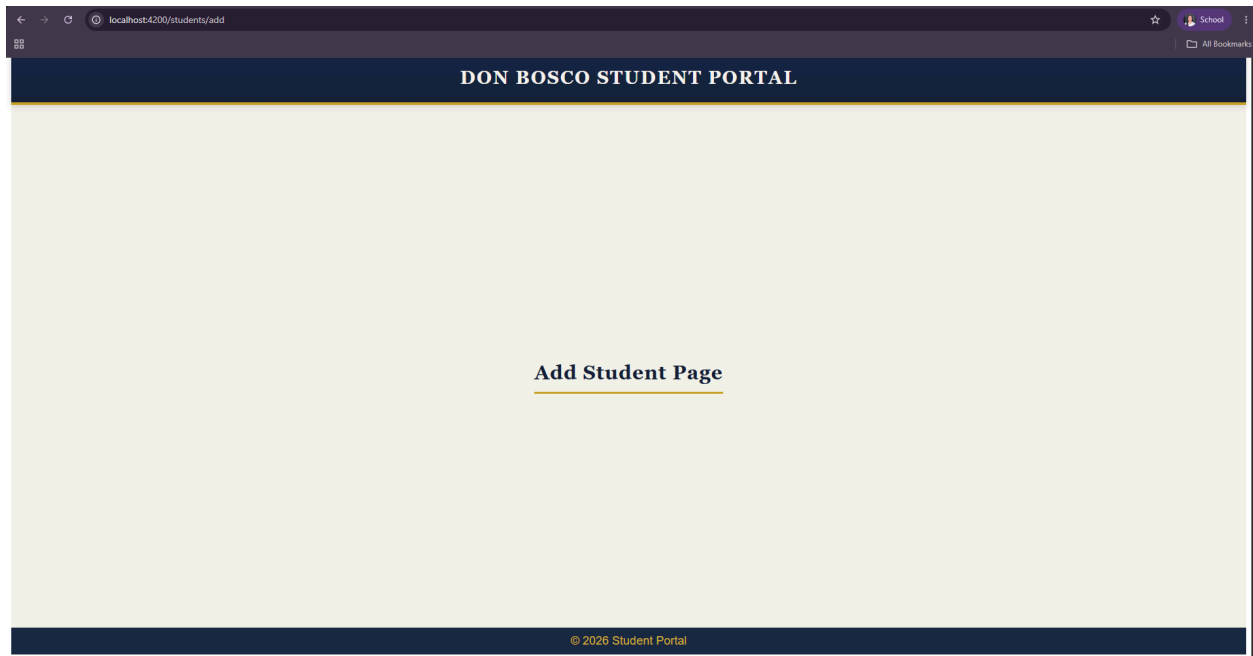
or

Add Student Page



Screenshot:

- The blank Add Student page displayed in the browser.



Flag 5 — Open the Portal Gate

Objective: Configure Angular Router.

Add routes for your three pages.

Your application should have routes similar to:

`/students`

`/students/:id`

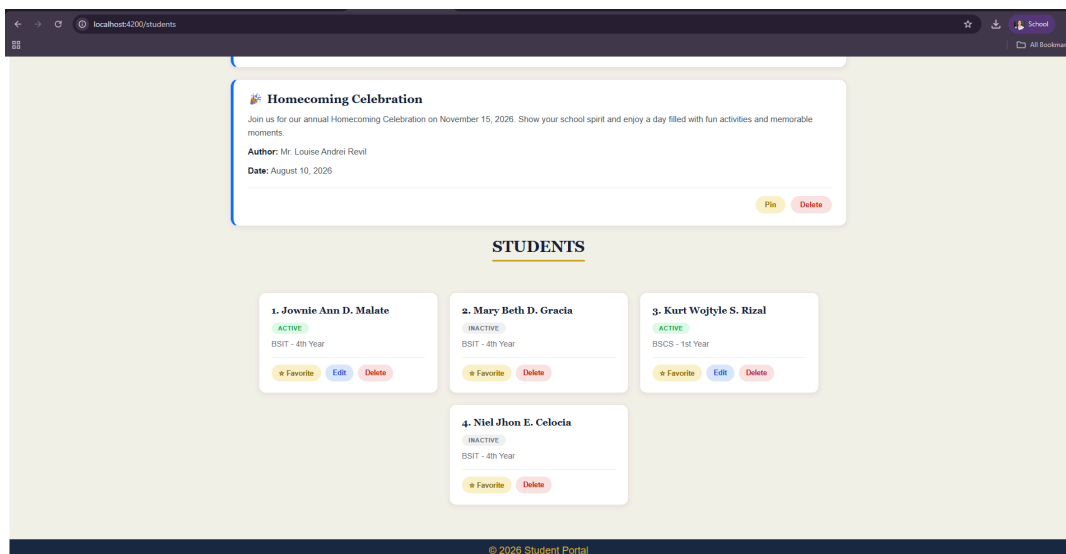
`/students/add`

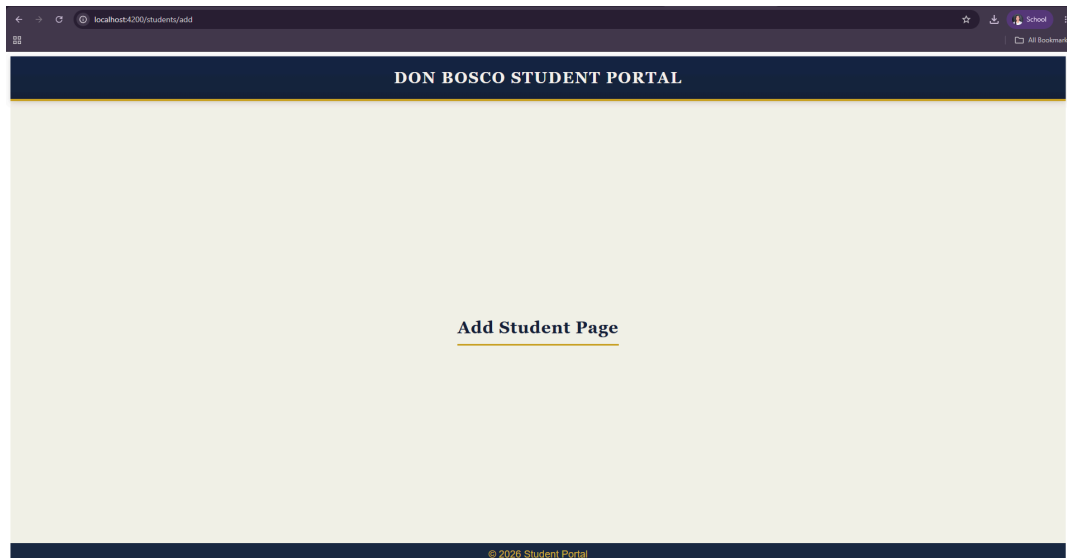
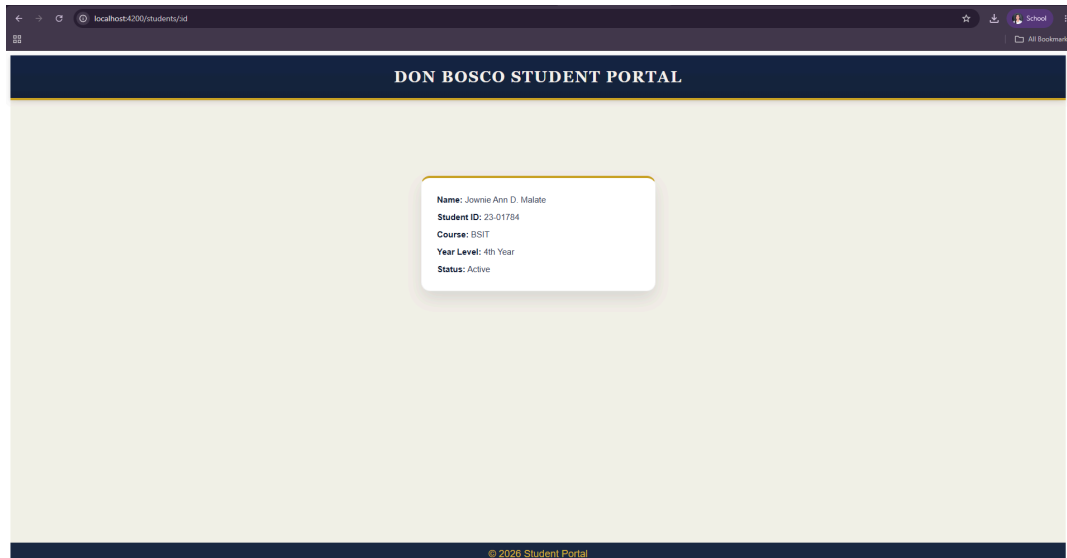
The exact route configuration may vary depending on your project structure.



Screenshot:

- Your route configuration shows the three routes.





Flag 6 — Navigate Without Reloading

Objective: Add navigation between pages.

Create navigation links or buttons that allow the user to move between:

- Student List
- Add Student
- Student Detail

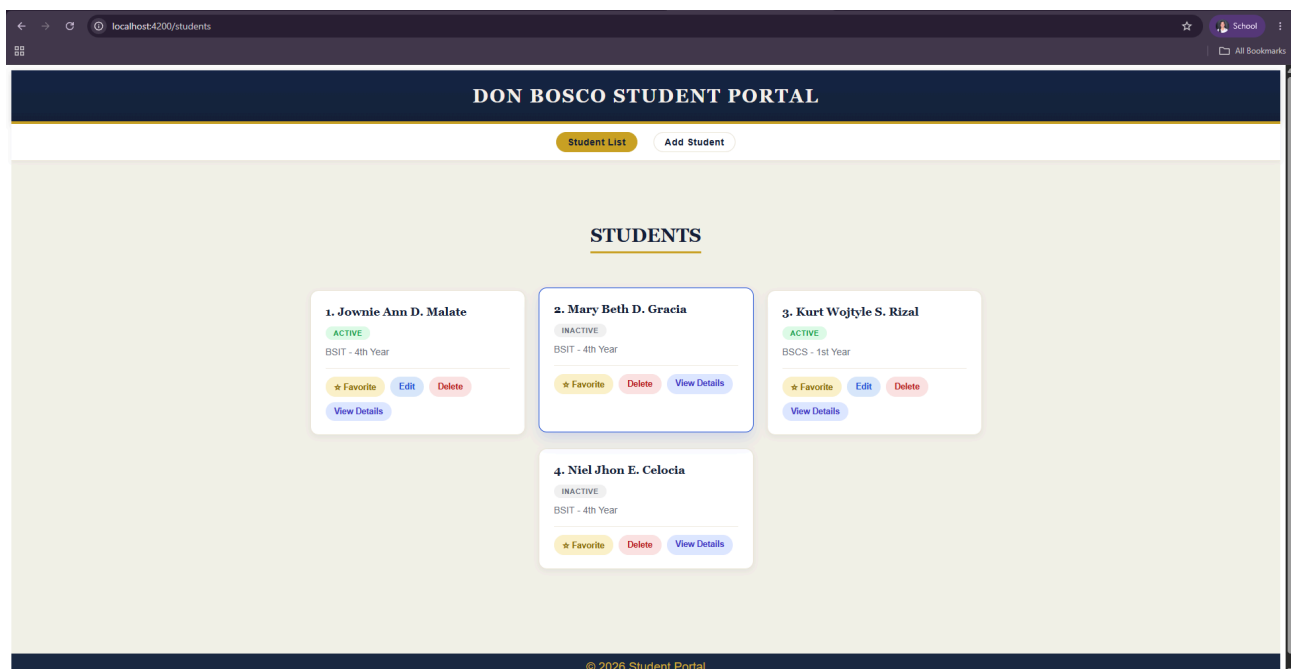
Use Angular's router navigation instead of manually changing the browser URL.

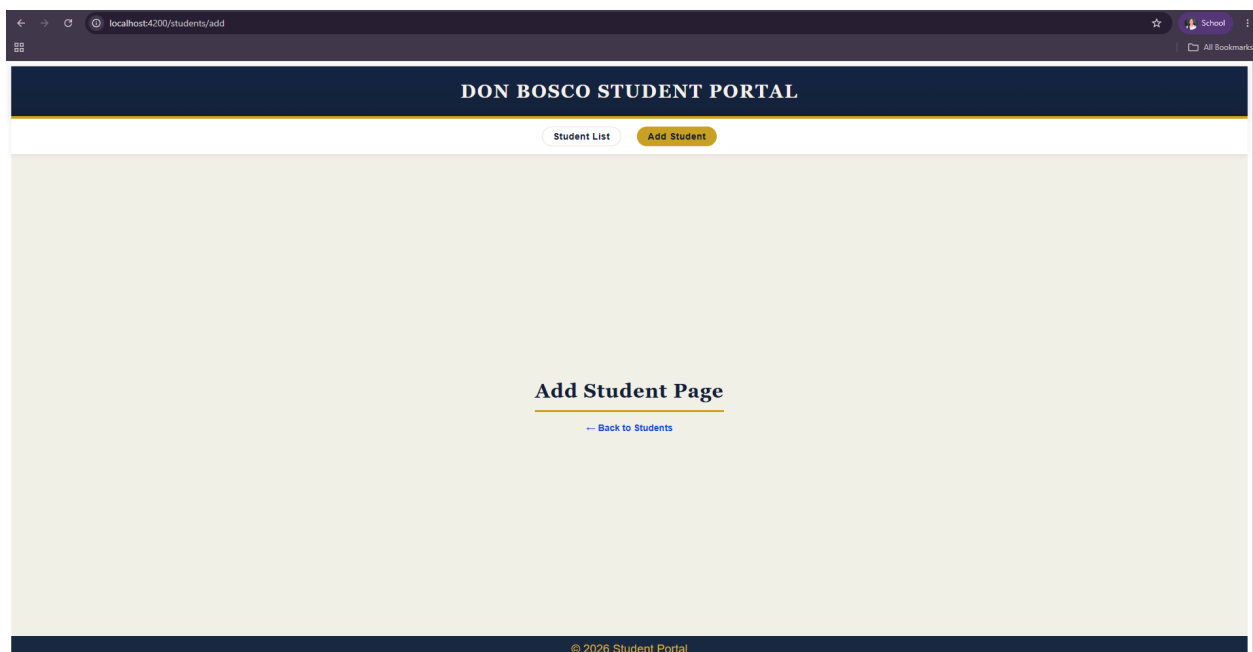
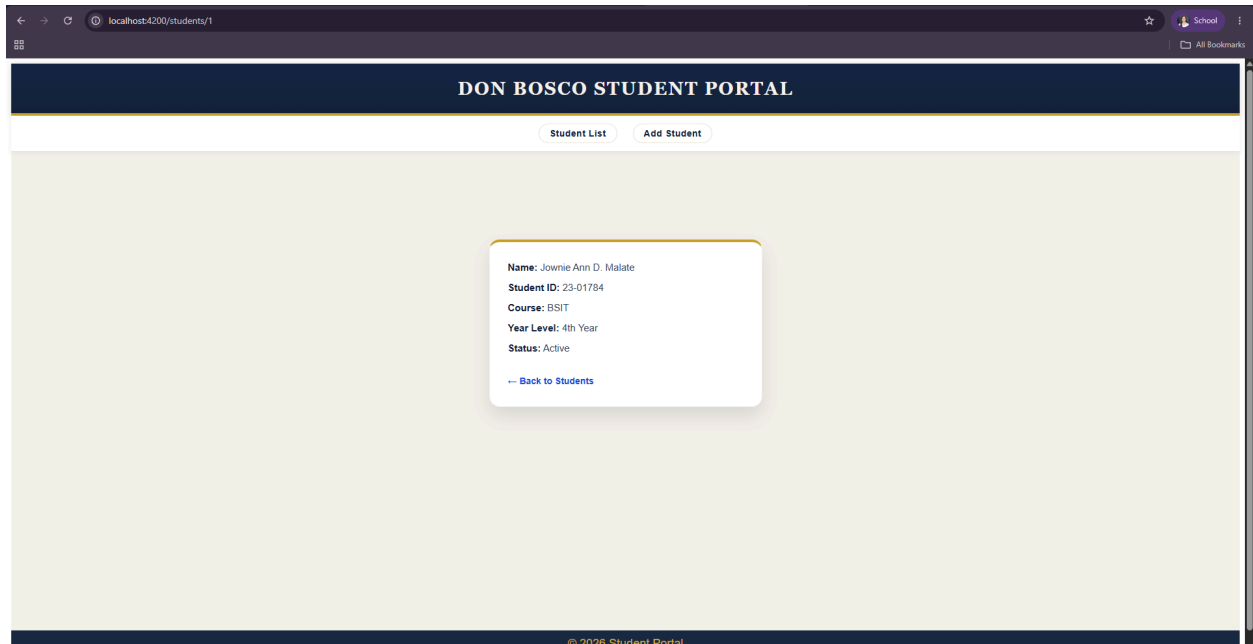
Hint: routerLink



Screenshot:

- Your navigation menu/buttons.
- The browser changes between pages without a full page reload.





Flag 7 — Give Every Student a Page

Objective: Navigate from a student card to its detail page.

Add a button or link to each student card.

Example:

View Student

When clicked, it should navigate to that student's detail route.

Example:

/students/1

/students/2

/students/3

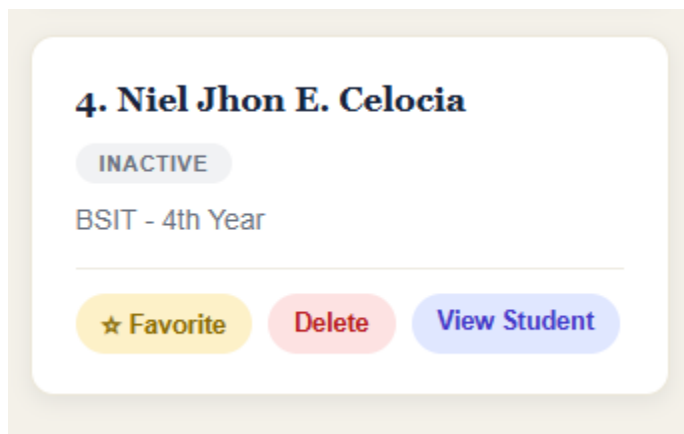
You don't need to load different student data yet. The goal is to successfully navigate using a route parameter.

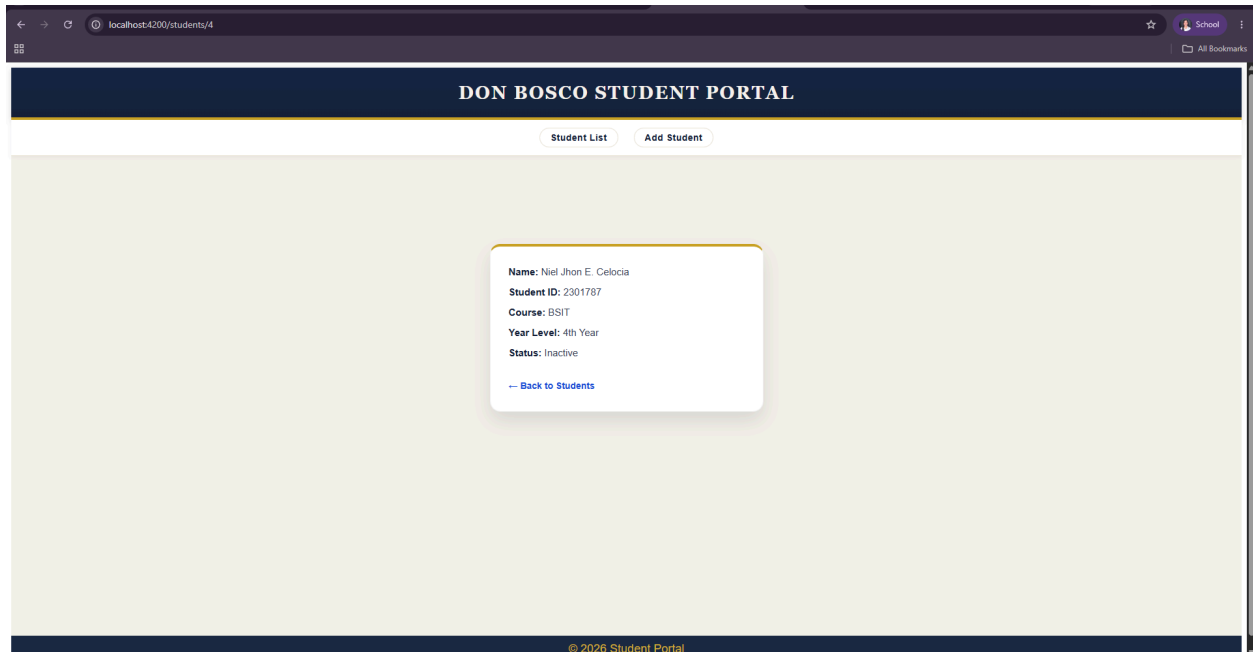
Hint: :id



Screenshot:

- A student card with the View Student button.
- The resulting Student Detail URL/page.





Flag 8 — Build the Portal Navigation

Objective: Make the Student Portal feel like a multi-page application.

Your final navigation should allow users to move between:

Student List



Student Detail

Student List



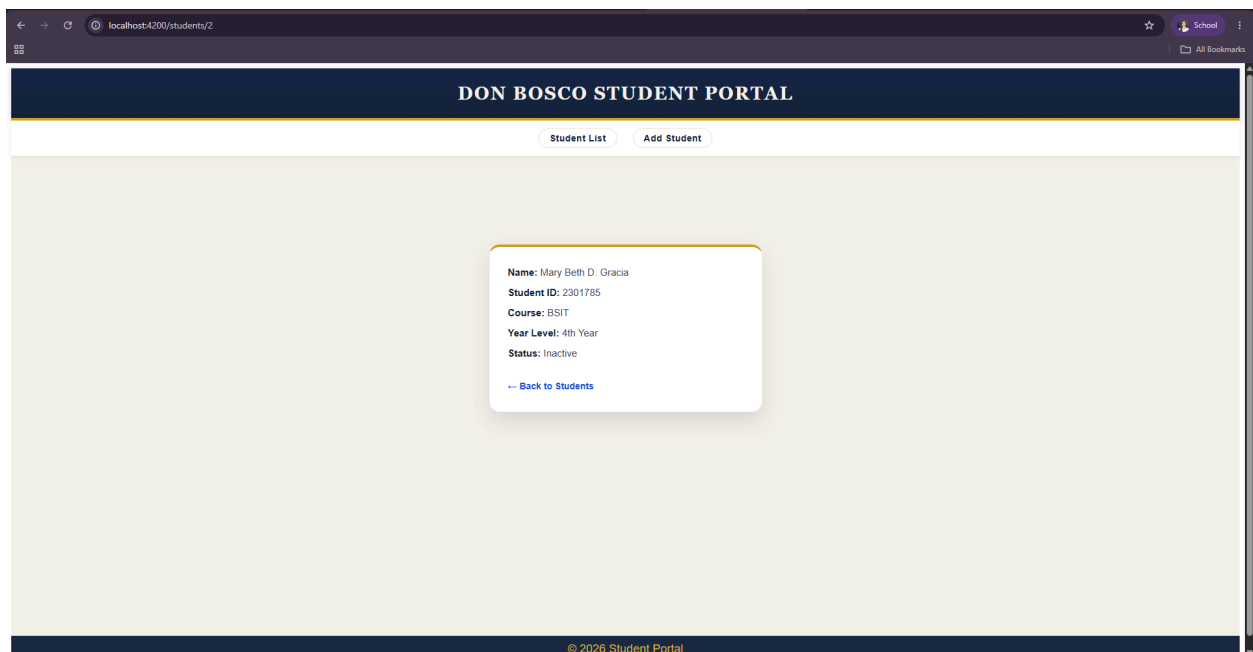
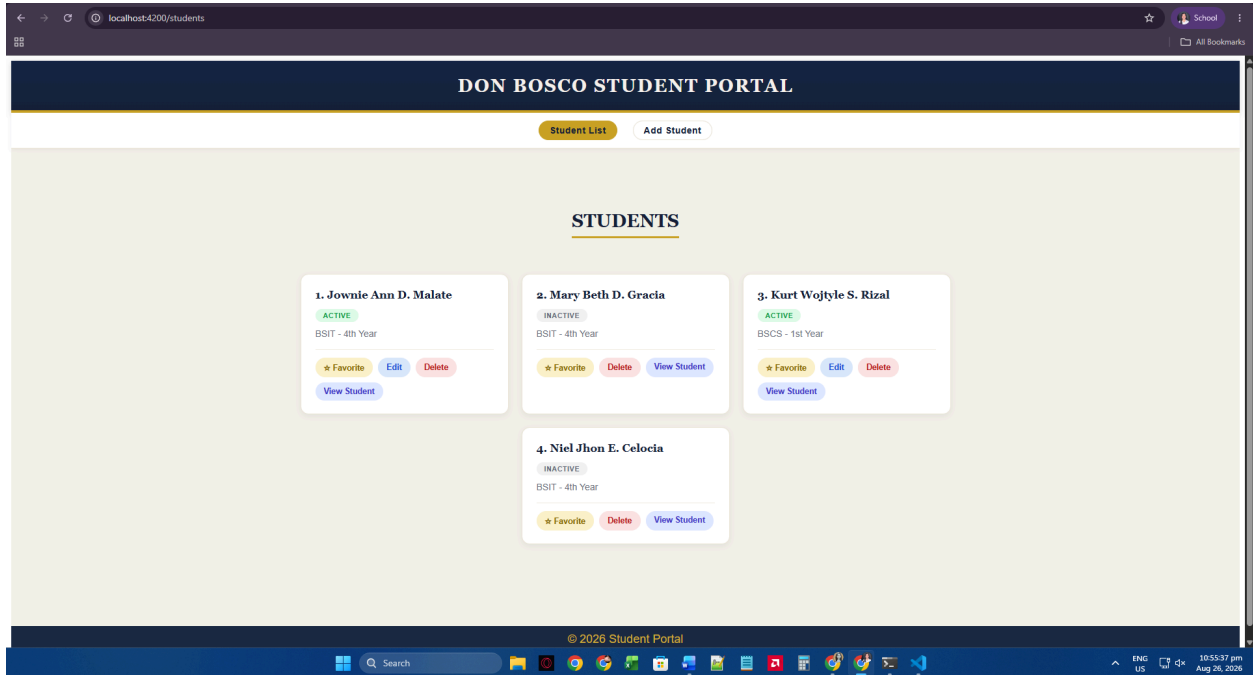
Add Student

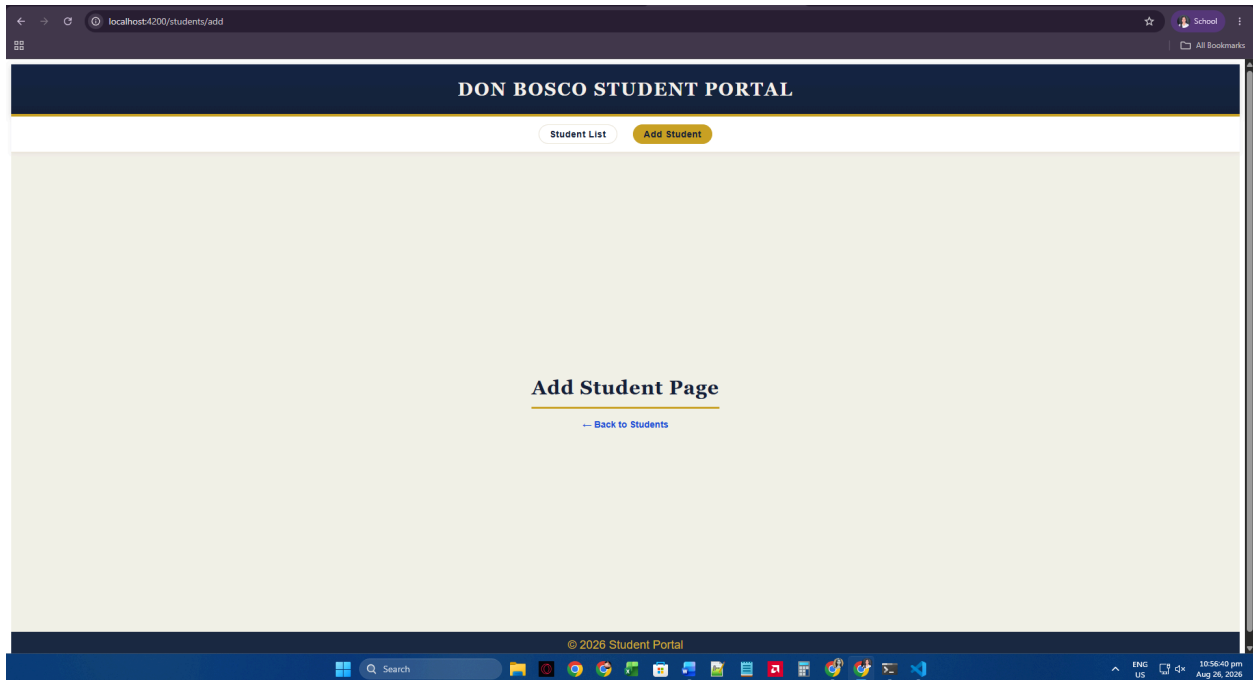
The Student List should remain accessible after navigating to other pages.



Screenshot:

- Your complete navigation works between all three pages.





Flag 9 — Commit Your Progress

Final Flag Screenshot

- Successful push to `feature/routing`.

```
loven@Izee MINGW64 ~/Documents/ITFE403A/ITFE403A_Pitogo (feature/routing)
$ git commit -m "CTF 7: Conditional UI"

loven@Izee MINGW64 ~/Documents/ITFE403A/ITFE403A_Pitogo (feature/routing)
$ git push -u origin feature/routing
info: please complete authentication in your browser...
Enumerating objects: 43, done.
Counting objects: 100% (43/43), done.
Delta compression using up to 8 threads
Compressing objects: 100% (21/21), done.
Writing objects: 100% (22/22), 3.42 KiB | 269.00 KiB/s, done.
Total 22 (delta 12), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (12/12), completed with 11 local objects.
To https://github.com/Nami19/ITFE403A_Pitogo.git
  4b7435c..82a2811 feature/routing -> feature/routing
branch 'feature/routing' set up to track 'origin/feature/routing'.
```